LOSSLESS HUFFMAN FILE COMPRESSOR

Design, Implementation and Analysis of a C++ Compressor & Extractor

Report type: Technical project report

Language: C++ (STL, binary file I/O)

Components: `library.h`, `compress.cpp`, `extract.cpp`

Contents

1. Abstract 3
2. Background and Theory 3
2.1 Lossless versus lossy compression 3
2.2 Fixed-length versus variable-length codes 3
2.3 Shannon entropy 3
2.4 The prefix-free property 3
2.5 The Huffman tree 4
2.6 Greedy strategy and the min-heap 4
3. System Design 4
3.1 Overview 4
3.2 Compressed file format 4
3.3 Deliberately minimal data types 5
4. Data Structures 5
5. Algorithms 6
5.1 Huffman tree construction 6
5.2 Code generation (DFS) 6
5.3 Bit packing 6
5.4 Prefix-free tree-walk decoding 6
5.5 Bitwise formulas used 6
6. C++ Features Used 6
7. Source Code 7
7.1 library.h 7
7.2 compress.cpp 9
7.3 extract.cpp 10
8. Build and Usage 11
9. Testing and Results 12
9.1 Observations 12
10. Complexity Analysis 12
11. Limitations and Possible Improvements 13
12. Conclusion 13
Appendix A: Frequently Asked Questions 13

1. Abstract

This report documents a lossless file compressor and extractor built on Huffman coding. The program reads any file as a stream of raw bytes, counts how often each byte value occurs, builds an optimal prefix-free binary tree from those counts, and rewrites the file using short codes for frequent bytes and longer codes for rare ones. A matching extractor rebuilds the identical tree from a stored frequency table and restores the original file byte for byte.

The implementation deliberately uses a minimal set of types (int, char, string and Node*), three small source files, and the C++ Standard Template Library. Round-trip tests on text, source code, binary, single-symbol and empty inputs all reproduced the originals exactly (Section 9).

2. Background and Theory

2.1 Lossless versus lossy compression

Data compression reduces the number of bits needed to represent information. Lossless methods allow perfect reconstruction of the original data and are essential for source code, executables, documents, archives and databases. Lossy methods (MP3, MP4, JPEG) discard imperceptible information for higher ratios and cannot restore the original exactly. This project is lossless.

2.2 Fixed-length versus variable-length codes

Standard ASCII spends exactly 8 bits on every character, whether it appears once or a million times. In natural language, symbol frequencies are heavily skewed, so fixed-length coding wastes space. Variable-length coding assigns short codes (roughly 1 to 3 bits) to frequent symbols and longer codes (roughly 6 to 10 bits) to rare ones, which minimises the average bits per character across the file.

2.3 Shannon entropy

In 1948 Claude Shannon showed that every data source has a theoretical compression limit, its entropy:

$$H = -\Sigma\, P(x_i) \cdot \log_2 P(x_i) \quad \textit{(bits per character)}$$

where $P(x_i)$ is the probability (frequency divided by total) of symbol x_i. Huffman coding, invented by David Huffman in 1952, produces an optimal prefix code whose average length is within 1 bit of this entropy limit.

2.4 The prefix-free property

Variable-length codes raise the question of where one code ends and the next begins. Consider the ambiguous code A=0, B=1, C=01: the stream “01” could mean AB or C. Huffman codes avoid this because no codeword is a prefix of another. Symbols live only at the leaves of the tree, so the decoder can walk the bitstream left to right in a single pass and emit a symbol the instant it reaches a leaf, with no delimiters and no backtracking.

2.5 The Huffman tree

The Huffman tree is a full binary tree: every node has either zero children (leaf) or exactly two (internal). For K distinct symbols there are K leaves, K − 1 internal nodes and 2K − 1 nodes in total, so at most 511 nodes for byte data (K ≤ 256).

- **Leaf nodes** hold a byte value and its frequency.
- **Internal nodes** hold the sum of their children's frequencies and act as branches (0 = left, 1 = right).
- **Root** has a frequency equal to the total number of bytes in the file.

The algorithm minimises the weighted external path length $W = \Sigma\ (\text{frequency}_i \times \text{depth}_i)$. Frequent symbols sit near the root, so fewer bits are written overall.

2.6 Greedy strategy and the min-heap

Huffman's algorithm is greedy: at each step it merges the two lowest-weight trees. Pairing the rarest symbols deepest guarantees that the most common symbols end up closest to the root. A min-heap (std::priority_queue with a reversed comparator) retrieves the two smallest nodes in O(log K) instead of the O(K) needed for an unsorted array, reducing total tree-building cost from $O(K^2)$ to O(K log K).

3. System Design

3.1 Overview

Compression is a two-pass process. Pass 1 counts byte frequencies and a header is written. Pass 2 re-reads the input and emits the prefix code of every byte through a bit-packing writer. Extraction reads the header, rebuilds the same tree using the same function, and decodes the bitstream by walking the tree.

Stage	Compressor	Extractor
1	Pass 1: count frequencies into freq[256]	Read totalBytes and freq[256] from header
2	Write header (totalBytes + freq table)	buildTree(freq): identical tree
3	buildTree(freq) and generateCodes()	Read bits with BitReader, walk tree
4	Pass 2: write each byte's code via BitWriter	Emit byte at each leaf, reset to root
5	flush() pads final byte with zeros	Stop when decodedBytes == totalBytes

3.2 Compressed file format

Region	Size	Content
Header: size	4 bytes	totalBytes, the original file length (an int)
Header: table	1024 bytes	freq[256], 256 four-byte frequency counts

Region	Size	Content
Payload	variable	Concatenated prefix codes, MSB first, zero-padded to a byte boundary

Storing totalBytes solves the trailing-padding problem. A stream needing 27 bits must occupy 4 bytes (32 bits), leaving 5 padding bits that a naive decoder might turn into false characters. The extractor stops the instant decodedBytes reaches totalBytes, so padding is ignored.

3.3 Deliberately minimal data types

The codebase uses only four basic types, to keep it simple and avoid sign and width pitfalls:

Type	Role	Why
int	Byte values, counts, sizes, bit buffers, loop indices	Avoids unsigned char / uint64_t / size_t; ch & 255 prevents negative indices
char	Raw file stream operations (get / put)	C++ streams read and write 1-byte chars
string	Variable-length codes such as "1101"	Easy concatenation while walking the tree
Node*	Links tree nodes together	Builds the binary tree

4. Data Structures

Structure	Description
struct Node	Tree node with fields ch, freq, left (bit 0) and right (bit 1). Constructor Node(int c, int f, Node* l = nullptr, Node* r = nullptr) uses a member initializer list and default arguments.
struct Compare	Functor overloading operator(); returns a->freq > b->freq so std::priority_queue behaves as a min-heap.
struct BitWriter	Holds an ofstream&, an 8-bit buffer and a bit count. Bridges variable-length codes and byte-oriented disk I/O.
struct BitReader	Holds an ifstream&, the current byte buffer and remaining-bit count. Dispenses single bits to the decoder.
int freq[256]	Direct-address frequency table: index = byte value, giving O(1) lookup and update.
string codes[256]	Lookup table mapping each byte value to its prefix code string.

5. Algorithms

5.1 Huffman tree construction

1. Create a leaf node for every byte value with freq > 0 and push it into the min-heap.
2. While the heap holds more than one node: pop the two smallest nodes a and b, create a parent with freq = a->freq + b->freq (left = a, right = b), and push it back.
3. The single remaining node is the root of the Huffman tree.

Edge cases handled: an empty frequency table returns nullptr; a single unique byte gets a synthetic parent so the symbol still receives a one-bit code ("0").

5.2 Code generation (DFS)

A depth-first traversal from the root appends “0” when moving left and “1” when moving right. On reaching a leaf, the accumulated path is stored in codes[leaf->ch]. Cost is O(K).

5.3 Bit packing

Hardware writes whole bytes, so codes are accumulated in an 8-bit register using buffer = (buffer << 1) | bit. When eight bits are collected the byte is written to disk. At end of file, flush() left-shifts the remaining bits by (8 − count) to pad with trailing zeros.

5.4 Prefix-free tree-walk decoding

1. Set curr to the root.
2. Read one bit; move to curr->left for 0 or curr->right for 1.
3. If curr is a leaf, output its byte and reset curr to the root.
4. Repeat until decodedBytes equals totalBytes.

Decoding is unambiguous because symbols exist only at leaves.

5.5 Bitwise formulas used

Expression	Purpose
ch & 255	Masks the low 8 bits so a signed char becomes 0 to 255
(buffer << 1) | (bit & 1)	Makes room and inserts the incoming bit at position 0
(buffer >> count) & 1	Right-shifts the wanted bit to position 0 and isolates it
buffer <<= (8 - count)	Aligns leftover bits to the high end and pads with zeros

6. C++ Features Used

- **Header guards** (#ifndef / #define / #endif) prevent duplicate definitions when library.h is included.

- **Structs with constructors,** member initializer lists and default arguments for compact, safe initialisation.
- **Functors and operator overloading** (Compare::operator()) to customise std::priority_queue.
- **STL:** <queue> and <vector> for the heap, <string> for code strings.
- **Pass by reference:** BitWriter and BitReader bind to existing streams, avoiding illegal stream copies.
- **Binary file I/O:** ios::binary disables newline translation so any file type is processed raw; get/put, read/write, clear and seekg(0, ios::beg) support the two-pass design.
- **inline functions** allow buildTree and generateCodes to live in a header without multiple-definition errors.
- **Range-based for loops and the ternary operator** for concise iteration and tree traversal.

7. Source Code

The complete implementation follows. It consists of one shared header and two programs.

7.1 library.h

Shared types and algorithms: Node, Compare, BitWriter, BitReader, buildTree and generateCodes.

Listing 1: library.h

```
#ifndef LIBRARY_H
#define LIBRARY_H

#include <iostream>
#include <fstream>
#include <vector>
#include <queue>
#include <string>

using namespace std;

// Huffman Binary Tree Node
struct Node {
    int ch;         // Character byte value (0 to 255)
    int freq;       // Occurrence count of the character
    Node *left;     // Left child (bit '0')
    Node *right;    // Right child (bit '1')

    Node(int c, int f, Node* l = nullptr, Node* r = nullptr)
        : ch(c), freq(f), left(l), right(r) {}
};

// Min-Heap comparator (smallest frequency gets highest priority)
struct Compare {
    bool operator()(Node* a, Node* b) {
        return a->freq > b->freq;
    }
};

// Packs individual bits into 8-bit bytes before writing to disk
```

```
struct BitWriter {
    ofstream& out;
    int buffer;     // 8-bit accumulator
    int count;      // Bits currently in buffer (0 to 8)

    BitWriter(ofstream& os) : out(os), buffer(0), count(0) {}

    void writeBit(int bit) {
        buffer = (buffer << 1) | (bit & 1); // Insert bit at the rightmost position
        count++;
        if (count == 8) { // Buffer is full -> write 1 byte to file
            out.put((char)buffer);
            buffer = 0;
            count = 0;
        }
    }

    void writeCode(const string& code) {
        for (char c : code) {
            writeBit(c - '0'); // Convert '0'/'1' to 0/1
        }
    }

    // Flush any leftover bits with trailing zero padding
    void flush() {
        if (count > 0) {
            buffer <<= (8 - count);
            out.put((char)buffer);
            buffer = 0;
            count = 0;
        }
    }
};

// Reads 1 byte from disk and dispenses individual bits (0 or 1)
struct BitReader {
    ifstream& in;
    int buffer;     // Current byte being read
    int count;      // Remaining unread bits in current byte

    BitReader(ifstream& is) : in(is), buffer(0), count(0) {}

    int readBit() {
        if (count == 0) { // All bits read -> fetch next byte from file
            char c;
            if (!in.get(c)) return -1; // End of file
            buffer = c & 255;          // Ensure safe 0-255 range
            count = 8;
        }
        count--;
        return (buffer >> count) & 1; // Extract highest available bit
    }
};

// Builds the Huffman Tree from character frequencies using a Min-Heap
inline Node* buildTree(const int freq[256]) {
    priority_queue<Node*, vector<Node*>, Compare> pq;

    // Push all present characters as leaf nodes
```

```
    for (int i = 0; i < 256; i++) {
        if (freq[i] > 0) {
            pq.push(new Node(i, freq[i]));
        }
    }

    if (pq.empty()) return nullptr;

    // Handle edge case: single unique character
    if (pq.size() == 1) {
        Node* only = pq.top(); pq.pop();
        return new Node(0, only->freq, only, nullptr);
    }

    // Repeatedly merge 2 lowest-frequency nodes until only root remains
    while (pq.size() > 1) {
        Node* a = pq.top(); pq.pop();
        Node* b = pq.top(); pq.pop();
        pq.push(new Node(0, a->freq + b->freq, a, b));
    }

    return pq.top(); // Root of the Huffman Tree
}

// Generates prefix codes via DFS (Left = '0', Right = '1')
inline void generateCodes(Node* root, string codes[256], string code = "") {
    if (!root) return;
    if (!root->left && !root->right) { // Leaf node reached
        codes[root->ch] = code.empty() ? "0" : code;
        return;
    }
    generateCodes(root->left, codes, code + "0");
    generateCodes(root->right, codes, code + "1");
}

#endif
```

7.2 compress.cpp

Two-pass compressor: count frequencies, write the header, build the tree, then encode the data.

Listing 2: compress.cpp

```
#include "library.h"

int main(int argc, char* argv[]) {
    if (argc != 3) {
        cout << "Usage: ./compress <input_file> <output_file.huff>\n";
        return 1;
    }

    ifstream input(argv[1], ios::binary);
    ofstream output(argv[2], ios::binary);

    if (!input.is_open() || !output.is_open()) {
        cerr << "Error opening files.\n";
        return 1;
```

```
    }

    // Step 1: Count character frequencies (Pass 1)
    int freq[256] = {0};
    int totalBytes = 0;
    char ch;

    while (input.get(ch)) {
        freq[ch & 255]++;
        totalBytes++;
    }

    // Step 2: Write header (Original size + Frequency table)
    output.write((char*)&totalBytes, sizeof(int));
    output.write((char*)freq, sizeof(freq));

    if (totalBytes == 0) {
        cout << "Compressed empty file.\n";
        return 0;
    }

    // Step 3: Build Huffman Tree & generate prefix codes
    Node* root = buildTree(freq);
    string codes[256];
    generateCodes(root, codes);

    // Step 4: Encode file content (Pass 2)
    input.clear();
    input.seekg(0, ios::beg);
    BitWriter bw(output);

    while (input.get(ch)) {
        bw.writeCode(codes[ch & 255]);
    }
    bw.flush();

    cout << "Compressed " << argv[1] << " -> " << argv[2] << " (" << totalBytes << "
bytes)\n";
    return 0;
}
```

7.3 extract.cpp

Reads the header, rebuilds the identical tree and decodes the bitstream.

Listing 3: extract.cpp

```
#include "library.h"

int main(int argc, char* argv[]) {
    if (argc != 3) {
        cout << "Usage: ./extract <input_file.huff> <output_file>\n";
        return 1;
    }

    ifstream input(argv[1], ios::binary);
    ofstream output(argv[2], ios::binary);
```

```
    if (!input.is_open() || !output.is_open()) {
        cerr << "Error opening files.\n";
        return 1;
    }

    // Step 1: Read header (Original size + Frequency table)
    int totalBytes = 0;
    int freq[256] = {0};

    input.read((char*)&totalBytes, sizeof(int));
    input.read((char*)freq, sizeof(freq));

    if (totalBytes == 0) {
        cout << "Extracted empty file.\n";
        return 0;
    }

    // Step 2: Reconstruct the identical Huffman Tree
    Node* root = buildTree(freq);
    Node* curr = root;

    // Step 3: Decode bitstream by traversing tree
    BitReader br(input);
    int decodedBytes = 0;

    while (decodedBytes < totalBytes) {
        int bit = br.readBit();
        if (bit == -1) break;

        // Traverse: 0 = left child, 1 = right child
        curr = (bit == 0) ? curr->left : curr->right;

        // Leaf node reached -> Output decoded character
        if (!curr->left && !curr->right) {
            output.put((char)curr->ch);
            decodedBytes++;
            curr = root; // Reset to root for next character
        }
    }

    cout << "Extracted " << argv[1] << " -> " << argv[2] << " (" << decodedBytes << "
bytes)\n";
    return 0;
}
```

8. Build and Usage

Listing 4: Build and run

```
g++ -O2 -o compress compress.cpp
g++ -O2 -o extract extract.cpp

./compress  input.txt  input.txt.huff
./extract   input.txt.huff  restored.txt
```

```
cmp input.txt restored.txt   # no output means identical
```

The -O2 flag enables optimisation level 2 (function inlining, better register allocation), which speeds up the bit-level loops considerably.

9. Testing and Results

Each file below was compressed and then extracted, and the result was compared byte for byte with the original using cmp. All seven round trips succeeded. Sizes are in bytes and include the fixed 1,028-byte header.

Test file	Original	Compressed	Ratio	Round trip
huffman_devnotes.txt (text)	14,194	10,163	71.6%	OK
huffman_theories.txt (text)	9,678	7,132	73.7%	OK
theories text repeated 20x	193,560	123,101	63.6%	OK
library.h (source code)	3,514	3,074	87.5%	OK
50,000 random bytes	50,000	51,028	102.1%	OK
1,000 × the letter ‘a’	1,000	1,153	115.3%	OK
empty file	0	1,028	n/a	OK

9.1 Observations

- Text compresses well because its byte frequencies are skewed. The ratio improves as files grow, because the 1,028-byte header is amortised over more data (63.6% for the 193 KB file versus 73.7% for the 9.7 KB file).
- Random data has a near-uniform distribution, so there is no redundancy to exploit and the output is slightly larger than the input by roughly the header size.
- The single-symbol file shows the edge case working: each byte becomes one bit (125 payload bytes for 1,000 symbols), but the header makes the total larger than the original.
- The empty file produces a header-only output and extracts correctly.

10. Complexity Analysis

Let N be the number of bytes in the file and K the number of distinct byte values (K ≤ 256).

Phase	Time	Notes
Pass 1: frequency counting	O(N)	One array increment per byte
Build min-heap and Huffman tree	O(K log K)	At most 256 log 256, effectively constant

Phase	Time	Notes
Generate codes (DFS)	O(K)	Visits at most 511 nodes
Pass 2: encoding	O(N)	One code lookup and bit writes per byte
Decoding	O(N)	One tree step per bit
Overall	O(N)	Linear in file size

Space: auxiliary memory is O(1): a 1,024-byte frequency array, a tree of at most 511 nodes and an 8-bit buffer. Input and output are streamed, so file size does not affect memory use.

11. Limitations and Possible Improvements

- **Fixed header overhead.** Storing all 256 four-byte counts costs 1,024 bytes regardless of the data. Storing only non-zero entries, or the tree shape and code lengths (canonical Huffman), would help small files.
- **Size limit.** totalBytes and the counts are 32-bit int values, so very large files (about 2 GB or more) would overflow them.
- **No incompressible-data fallback.** Random or already-compressed input grows slightly; a "stored" mode could copy such files unchanged.
- **Memory management.** Tree nodes are allocated with new and never deleted; this is harmless for a short-lived command-line tool but a recursive free routine would make it leak-free.
- **Byte-level model only.** Huffman coding on single bytes cannot exploit repeated phrases; dictionary methods (LZ77, Deflate) combine both ideas for stronger results.
- **Robustness.** The extractor trusts the header; adding a magic number and checksum would detect corrupt or non-Huffman files.

12. Conclusion

The project implements a complete, correct and compact lossless compressor. It applies a greedy algorithm (Huffman), a min-heap, a full binary tree, DFS code generation and bit-level I/O in about 230 lines of C++. Theory predicts near-entropy compression with linear running time and constant auxiliary memory, and the tests confirm lossless round trips across text, source code, random binary, single-symbol and empty inputs. The main practical weakness, the fixed 1,028-byte header, is well understood and can be reduced with a canonical Huffman header.

Appendix A: Frequently Asked Questions

Q: Which data structures are used?
A: A binary tree (the Huffman tree), a min-heap (std::priority_queue) for building it, and a 256-element array for O(1) frequency counting.

Q: What are the time and space complexities?

A: Time is O(N), linear in file size. Auxiliary space is O(1) because the tree has at most 511 nodes.

Q: How does the extractor rebuild the tree without the original data?

A: The frequency table is stored in the header, and the extractor calls the exact same buildTree(freq) function.

Q: What is a prefix-free code?

A: No codeword is the start of another codeword, so the bitstream can be decoded unambiguously, bit by bit, without delimiters.

Q: Why must tie-breaking be deterministic?

A: Both programs must build an identical tree. They do, because both call the same buildTree with the same frequency array in the same order.

Q: What does -O2 do?

A: It enables optimisation level 2, including function inlining and register allocation, which makes the bit processing loops much faster.